Versi 2.3

Agustus 2026

PEMROGRAMAN FUNGSIONAL

MODUL 2 - Effect-free Programming and Declarative Paradigm

Penulis:

Fera Putri Ayu L., S.Kom., M.T.

HERDIANA DWI MAHARANI

NUR INDAH AMELIA AGUSTIN

LABORATORIUM INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MALANG

DAFTAR ISI

DAFTAR ISI **2**
PENDAHULUAN **3**
TUJUAN PRAKTIKUM 3
TARGET MODUL 3
PERSIAPAN 3
KEYWORDS 3
1. EFFECT-FREE PROGRAMMING **4**
2. PURE FUNCTION **4**
3. IMMUTABILITY **8**
3.1. MASALAH PADA DATA MUTABLE 9
3.2. TEKNIK MENGELOLA IMMUTABILITY DENGAN COPY 11
3.2.1. SHALLOW COPY VS DEEP COPY 12
4. DECLARATIVE PROGRAMMING **15**
5. ITERATOR & GENERATOR **19**
5.1. KONSEP ITERATOR 19
5.2. LIST COMPREHENSION 21
5.3. GENERATOR EXPRESSION 24
5.4. GENERATOR 26
6. LAZY EVALUATION **28**
CODE ANALYSIS **30**
ASSIGNMENT **31**
ASSIGNMENT 1 31
ASSIGNMENT 2 31
ASSESSMENT RUBRIC **32**
Detailed Assessment Rubric 33

PENDAHULUAN

TUJUAN PRAKTIKUM

1. Sub-CPMK 1: Mahasiswa mampu menganalisis konsep dasar paradigma Functional Programming dalam konteks komputasi data untuk menghasilkan perangkat lunak yang berkualitas.
2. Sub-CPMK 2: Mahasiswa mampu mengevaluasi dan memilih standar metode Functional Programming yang sesuai berdasarkan karakteristik permasalahan perangkat lunak.

TARGET MODUL

1. Mahasiswa memahami dan mengimplementasikan konsep dasar Functional Programming (Effect-Free Programming) dengan menggunakan teknik pengelolaan immutability.

2. Mahasiswa mampu mengimplementasikan teknik Lazy Evaluation menggunakan iterator dan generator.

3. Mahasiswa mampu menentukan teknik yang sesuai untuk menyelesaikan permasalahan pemrograman perangkat lunak berbasis Functional Programming.

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum praktikum.

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi
3. Google Collab/VS Code/Jupyter Notebook
4. *Source Code Modul 2*

KEYWORDS

Pure Function, Declarative Programming, Lazy Evaluation, Generator, Exception Handling

1. EFFECT-FREE PROGRAMMING

Dalam paradigma Functional Programming, Effect-Free Programming merupakan pendekatan dasar yang menekankan penulisan fungsi tanpa menyebabkan perubahan pada state sistem di luar fungsi tersebut. Untuk memahami mengapa pendekatan effect-free penting, kita dapat menggunakan analogi matematika. Dalam matematika, sebuah fungsi didefinisikan sebagai suatu relasi yang memetakan nilai input ke nilai output. Fungsi ini biasanya dituliskan sebagai f(x).

Sebagai contoh sederhana:

$$f(x) = 2x + 10$$

Ketika kita memberikan nilai input x = 5, output yang dihasilkan akan selalu 20. Sifat ini disebut deterministik, yang berarti bahwa kapan pun dan di mana pun fungsi tersebut dijalankan, fungsi tersebut akan menghasilkan output yang sama selama nilai inputnya tidak berubah. Dalam kode program, pendekatan ini mengharuskan fungsi yang kita buat untuk tidak mengubah atau memanipulasi variabel global maupun argumen yang diteruskan melalui parameter. Sebaliknya, fungsi harus berfokus hanya pada pemrosesan data input untuk menghasilkan output baru.

Agar dapat mengikuti konsep Effect-Free Programming dengan baik, sebuah fungsi harus bekerja secara terisolasi. Artinya, fungsi tidak boleh melakukan hal-hal berikut:

1. Menimbulkan side effect tersembunyi, seperti perubahan state global.
2. Mengubah data yang berada di luar fungsi secara langsung atau melakukan operasi Input/Output (I/O) secara implisit, seperti mencetak sesuatu langsung ke console dari dalam logika utama perhitungan.

Sebaliknya, fungsi harus menangani data dengan hati-hati agar lingkungan di luar fungsi tidak berubah dan hasil yang dihasilkan dapat diprediksi. Untuk menerapkan konsep ini dalam pemrograman, paradigma fungsional menggunakan salah satu building block utama yang disebut **Pure Function**, yang akan dibahas pada bagian berikutnya.

2. PURE FUNCTION

Pure Function merupakan bentuk nyata atau implementasi langsung dari filosofi Effect-Free Programming dalam kode program. Karakteristik utama dari pure function adalah sifatnya yang deterministik, yaitu menjamin bahwa fungsi akan selalu menghasilkan output yang sama ketika menerima input yang sama, terlepas dari kapan atau di mana kode tersebut dijalankan. Selain itu, pure function bekerja secara terisolasi, yang berarti tidak akan menimbulkan side effect apa pun di dalam sistem, seperti mengubah variabel global atau memodifikasi data state di luar scope-nya. Seluruh prosesnya terbatas pada menerima input, melakukan perhitungan, dan secara konsisten mengembalikan nilai baru, seperti yang ditunjukkan pada contoh berikut:

Catatan: Kode sengaja dibagi menjadi beberapa cell untuk memisahkan fungsi dari implementasinya. Praktik ini mengikuti standar pemrograman yang umum digunakan, yaitu memisahkan program utama dari tahap pengujian untuk memudahkan proses evaluasi atau peninjauan.

```
'''
define the 'f' function that take any number 'x' as input and
return the calculation of '2x + 10'
'''
def f(x):
# Process the input & return the result without modifying the original 'x'
    return 2 * x + 10
    #return x + x + 10
```

Gambar 2.1 Definisi Pure Function f(x)

```
# Initialize a global variable 'x' with a value of 5
x = 5

# calling the 'f' function and store the result
result = f(x)

# print the Output: Result= 20
print("Result = ", result)

# print the value of x to prove it remains unchanged (Output: x = 5)
print("x = ", x)
```

```
Result =  20
x =  5
```

Gambar 2.2 Memanggil f(x) dan Memastikan x Tidak Berubah

Seperti yang dapat dilihat, fungsi f(x) tidak mengubah state di luar dirinya sendiri (x). Isolasi penuh ini merupakan inti dari Effect-Free Programming. Ketika fungsi dijalankan, fungsi hanya menerima nilai input, melakukan perhitungan, lalu mengembalikan hasilnya. Variabel global x yang asli tetap aman dan tidak berubah di memori. Karena tidak terdapat side effect tersembunyi, seperti perubahan variabel di luar fungsi secara tidak sengaja, fungsi tersebut bekerja seperti rumus matematika yang dapat diandalkan. Jika diberikan input yang sama, fungsi dijamin akan menghasilkan output yang sama, tidak peduli berapa kali atau kapan fungsi tersebut dijalankan.

Pendekatan ini membuat kode lebih mudah diprediksi dan jauh lebih mudah untuk di-debug. Karena fungsi hanya bergantung pada input dan output-nya sendiri,

fungsi dapat diuji secara terpisah tanpa perlu khawatir mengganggu bagian lain dari program secara tidak sengaja. Dengan menghindari ketergantungan pada variabel global atau shared state, kode menjadi lebih bersih, stabil, dan terhindar dari logic bug yang biasanya terjadi ketika beberapa fungsi mencoba mengubah data yang sama secara bersamaan.

Untuk melihat bagaimana konsep ini diterapkan pada kumpulan data, mari kita lihat contoh kedua. Pada contoh ini, kita akan mencoba menambahkan bonus 5 poin ke dalam sebuah list berisi nilai ujian mahasiswa.

```
# the data that we have
exam_scores = [70, 80, 85]
print("Original exam_scores:", exam_scores)
```

```
Original exam_scores: [70, 80, 85]
```

Gambar 2.3 Data exam_scores Awal

```
# A pure function that returns a new list without modifying the input list
def add_bonus(scores, bonus):
    final_scores = []
    for i in scores:
        final_scores.append(i + bonus)
    return final_scores
```

Gambar 2.4 Definisi Pure Function add_bonus()

```
# testing
print("Before:", exam_scores)

result = add_bonus(exam_scores, 5)

print("New result:", result)
print("Original remains unchanged:", exam_scores)
```

```
Before: [70, 80, 85]
New result: [75, 85, 90]
Original remains unchanged: [70, 80, 85]
```

Gambar 2.5 Pengujian Output add_bonus()

Perhatikan bahwa fungsi add_bonus() sama sekali tidak mengubah nilai asli variabel exam_scores. Karakteristik inilah yang mendefinisikan sebuah Pure Function. Ketika fungsi dijalankan, fungsi membuat list baru yang kosong bernama final_scores di dalam memori lokalnya, mengisinya dengan nilai hasil perhitungan, lalu mengembalikan objek baru tersebut ke program utama. List asli di luar fungsi tetap tidak tersentuh dan

aman dari perubahan data yang tidak disengaja. Karena fungsi hanya bergantung pada parameternya (scores dan bonus) serta tidak mengubah lingkungan di luar fungsi, fungsi tersebut memenuhi prinsip utama Effect-Free Programming.

Selanjutnya, mari kita lihat perbedaannya dengan fungsi berikut yang memanipulasi variabel di luar scope-nya menggunakan kata kunci global:

```python
# A function that modifies a global variable (Impure Function)
def add_bonus_global(bonus):
    global exam_scores
    for i in range(len(exam_scores)):
        exam_scores[i] += bonus
    return exam_scores
```

Gambar 2.6 Definisi Impure Function add_bonus_global()

```python
# testing
print("Before:", exam_scores)

result = add_bonus_global(5)

print("New result:", result)
print("Original exam_scores be:", exam_scores)
```

```
Before: [70, 80, 85]
New result: [75, 85, 90]
Original exam_scores be: [75, 85, 90]
```

Gambar 2.7 Pengujian Output add_bonus_global()

Perhatikan pernyataan global exam_scores di dalam fungsi. Perintah ini memberikan izin kepada fungsi add_bonus_global() untuk mengubah isi variabel global tersebut secara langsung. Akibatnya, data asli exam_scores ikut berubah. Fungsi seperti ini dikategorikan sebagai Impure Function karena melanggar prinsip isolasi dengan menimbulkan side effect yang mengubah state program di luar fungsi itu sendiri.

Penggunaan impure function dapat cukup berisiko. Ketika sebuah fungsi dapat mengubah variabel global secara tidak terduga, proses menemukan bug menjadi jauh lebih sulit, terutama ketika program semakin besar dan beberapa fungsi mulai mengubah data yang sama. Dengan beralih ke Pure Function, data tetap bersifat immutable (tidak dapat diubah dari luar), sehingga keseluruhan kode menjadi lebih aman, lebih mudah diprediksi, dan lebih mudah diuji.

Untuk penjelasan lebih lanjut mengenai Pure Function, kamu bisa menonton video berikut: Python Functions 1: Pure Functions.

📌 Exercise 1: functional analysis

```
def add_bonus2(bonus):
    for i in range(len(exam_scores)):
        exam_scores[i] = exam_scores[i] + bonus
    return exam_scores
```

Gambar 2.8 Definisi Fungsi

Lengkapi kode berikut dengan memanggil fungsi add_bonus2(), kemudian jalankan program sebanyak 2-3 kali dan amati perubahan pada result serta exam_scores.

```
# (Your NIM)
print("Before:", exam_scores)

result =

print("New result:", result)
print("Original exam_scores be:", exam_scores)
```

Gambar 2.9 Exercise 1

📝 Analysis Question 1

Mengapa add_bonus2() mengubah data asli exam_scores meskipun tidak menggunakan kata kunci global? Jelaskan hasil analisis kamu. Berdasarkan karakteristiknya, apakah fungsi add_bonus2() termasuk fungsi pure atau impure? Jelaskan alasannya.

3. IMMUTABILITY

Dalam paradigma Functional Programming, Immutability merupakan salah satu prinsip utama yang berjalan beriringan dengan Effect-Free Programming dan Pure Function. Immutability berarti bahwa setelah suatu data atau objek dibuat, state atau nilainya tidak dapat diubah atau dimodifikasi. Jika ingin mengubah data, kita tidak memperbarui objek yang sudah ada secara langsung, tetapi membuat objek baru dengan nilai yang telah diperbarui. Dengan memastikan data tetap konstan selama program berjalan, kita dapat menghindari side effect yang tidak terduga, membuat kode lebih aman saat dijalankan secara thread-safe, dan memiliki kontrol yang lebih baik atas bagaimana data mengalir dalam program. Namun, menerapkan Immutability dalam Python membutuhkan pemahaman yang baik tentang cara Python menangani data di balik layar. Untuk memahami mengapa struktur data immutable diperlukan, kita perlu terlebih dahulu melihat risiko yang dapat muncul saat menggunakan objek yang dapat diubah.

3.1. MASALAH PADA DATA MUTABLE

Sebelum mempelajari cara menjaga data agar tetap immutable (tidak dapat diubah), kita perlu memahami terlebih dahulu bagaimana Python mengelola memori untuk tipe data yang mutable, seperti List atau Dictionary. Banyak pemula mengira bahwa untuk menyalin data dari satu variabel ke variabel lain, mereka cukup menggunakan assignment statement biasa atau operator sama dengan (=). Mari kita lihat kode berikut:

```
#define some data
x = [1, 2]
y = x

#check the value and memory address of the variable
print("Value of x: ", x)
print("ID of x: ", id(x))
print("Value of y: ", y)
print("ID of y: ", id(y))
```

```
Value of x:  [1, 2]
ID of x:  140579549071424
Value of y:  [1, 2]
ID of y:  140579549071424
```

Gambar 3.1 Shallow Assignment (y = x) Berbagi ID Memori yang Sama

Dalam Python, assignment statement seperti `y = x` tidak membuat duplikat objek. Pernyataan tersebut hanya membuat referensi baru yang menunjuk ke alamat objek yang sama di memori. Hal ini dapat dibuktikan dengan memeriksanya menggunakan fungsi `id()`, karena kedua variabel akan menghasilkan alamat memori yang sama. Karena keduanya menunjuk ke satu objek yang sama, setiap perubahan yang dilakukan pada variabel y secara otomatis akan mengubah data pada variabel `x`. Perhatikan contoh berikut:

```
print("\nModifying the first element of variable y")
y[0] = 5
print("Value of y: ", y)
print("Value of x: ", x) # The value of x changes as well
```

```
Modifying the first element of variable y
Value of y:  [5, 2]
Value of x:  [5, 2]
```

Gambar 3.2 Memodifikasi y Juga Mengubah x

Karakteristik objek yang isinya dapat diubah setelah dibuat disebut mutability dan objek seperti ini disebut mutable object. Dalam Functional Programming, perilaku assignment seperti ini menjadi masalah karena dapat menimbulkan side effect yang berpotensi memengaruhi integritas data asli di luar fungsi. Untuk menghindari masalah tersebut, kita dapat membuat salinan objek terlebih dahulu sebelum melakukan perubahan. Untungnya, objek yang mutable seperti list, dictionary, dan set memiliki method bawaan copy() yang dapat digunakan untuk menyalin objek tanpa memerlukan modul eksternal. Objek hasil salinan akan memiliki alamat memori sendiri, sehingga kita dapat memodifikasinya tanpa secara langsung memengaruhi objek aslinya. Mari kita coba menyalin list x menggunakan method copy().

```
# using built-in copy() method to duplicate an object
y = x.copy()

#check the value and memory address of the variable after using copy()
print("Value of x: ", x)
print("ID of x: ", id(x))
print("Value of y: ", y)
print("ID of y: ", id(y))
```

```
Value of x:  [1, 2]
ID of x:  140579549071424
Value of y:  <built-in method copy of list object at 0x7fdb3a15c840>
ID of y:  140579526975600
```

Gambar 3.3 Menyalin dengan Method Bawaan copy() pada List

Output menunjukkan bahwa x dan y memiliki nilai yang sama, tetapi memiliki alamat memori yang berbeda. Artinya, y merupakan objek baru yang dibuat oleh method copy(), bukan referensi lain yang menunjuk ke x. Karena keduanya merupakan objek yang terpisah, perubahan yang dilakukan pada salah satu list tidak akan memengaruhi list lainnya. Pendekatan ini membantu mengurangi side effect yang tidak diinginkan saat bekerja dengan objek yang mutable. Dengan membuat salinan, kita dapat bekerja dengan data mutable dengan aman sambil mempertahankan state dari objek aslinya.

Method bawaan copy() merupakan cara yang praktis untuk menyalin objek mutable seperti list, dictionary, dan set. Namun, ketika bekerja dengan struktur data yang lebih kompleks, kita mungkin memerlukan mekanisme penyalinan yang lebih lanjut. Untuk mendukung kebutuhan tersebut, Python menyediakan modul bawaan bernama copy yang menawarkan cara tambahan untuk menyalin objek sekaligus menjaga integritas data.

3.2. TEKNIK MENGELOLA IMMUTABILITY DENGAN COPY

Sekarang mari kita kembali melihat masalah pada fungsi add_bonus_global() yang mengubah data asli exam_scores. Untuk mengatasi masalah ini, selain membuat list kosong di dalam fungsi, kita juga dapat menggunakan modul copy untuk menyalin data dari satu variabel ke variabel lain. Perhatikan kode berikut:

```
# import the copy module first
import copy
```

Gambar 3.4 Mengimpor Modul copy

```
# Pure function utilizing shallow copy to maintain immutability
def add_bonus_copy(scores, bonus):
    final_scores = copy.copy(scores)
    for i in range(len(final_scores)):
        final_scores[i] += bonus
    return final_scores
```

Gambar 3.5 Pure Function dengan Shallow Copy

```
# testing
print("Original exam_scores:", exam_scores)

result = add_bonus_copy(exam_scores, 20)

print("Result after shallow copy: ", result)
print("Original data: ", exam_scores)
```

```
Original exam_scores: [70, 80, 85]
Result after shallow copy:  [90, 100, 105]
Original data:  [70, 80, 85]
```

Gambar 3.6 Pengujian Output add_bonus_copy()

Dengan menggunakan copy.copy(scores): Python membuat objek list baru di dalam memori dengan isi yang sama persis seperti exam_scores. Karena keduanya sekarang merupakan dua objek yang berbeda di dalam memori, for loop di dalam fungsi hanya akan memanipulasi list baru (final_scores). Dengan demikian, data asli exam_scores di luar fungsi tetap aman dan immutability-nya tetap terjaga. Pada tahap ini, mungkin muncul pertanyaan di benak kamu: “Pada fungsi add_bonus() di bagian Pure Function, kita dapat mencegah perubahan pada data asli hanya dengan membuat list kosong final_scores = [] dan menjalankan for loop secara manual. Jika cara manual tersebut sudah berhasil, mengapa kita membutuhkan metode lain?”

Jawabannya berkaitan dengan efisiensi dan kompleksitas data. Bayangkan jika data yang kita miliki bukan hanya sebuah list satu dimensi sederhana yang berisi angka, tetapi data nested yang besar dan kompleks, seperti dictionary di dalam list atau list di dalam dictionary. Jika kita tetap menyalin data secara manual menggunakan for loop bertingkat, kode akan menjadi sangat panjang, terlalu rumit, dan lebih rentan terhadap bug.

Inilah alasan Python menyediakan modul bawaan bernama copy sebagai solusi untuk menduplikasi objek secara langsung di memori. Namun, apakah copy.copy() selalu bekerja dengan sempurna untuk setiap jenis struktur data? Ternyata tidak. Di sinilah kita menemukan keterbatasan copy.copy() yang umumnya dikenal sebagai Shallow Copy, sehingga nantinya kita membutuhkan Deep Copy, yang akan dibahas pada bagian berikutnya.

3.2.1. SHALLOW COPY VS DEEP COPY

Modul copy menyediakan dua teknik untuk melakukan duplikasi, yaitu Shallow Copy (menyalin pada level permukaan) dan Deep Copy (menyalin pada level permukaan hingga level nested). Perbedaan antara kedua teknik ini sangat penting dan baru terlihat ketika bekerja dengan objek nested/compound, seperti list di dalam list atau list di dalam dictionary.

a. Shallow Copy

Sebelum masuk ke kode, kita perlu memahami terlebih dahulu apa yang sebenarnya dilakukan oleh Shallow Copy. Ketika melakukan shallow copy, Python membuat objek container baru di dalam memori. Namun, alih-alih menduplikasi item yang ada di dalam container tersebut, Python hanya menyalin alamat referensi elemen-elemennya. Artinya, shallow copy hanya menduplikasi lapisan terluar dari struktur data.

Selama kita bekerja dengan data sederhana dan datar (seperti daftar satu dimensi yang berisi integer), shallow copy bekerja dengan baik untuk memisahkan data. Namun, ketika berhadapan dengan objek nested atau compound, seperti list yang berisi list lainnya, shallow copy dapat menimbulkan bug yang sulit ditemukan karena lapisan bagian dalam dari list asli dan list hasil salinan tetap menunjuk ke lokasi yang sama di memori. Perhatikan kode berikut:

```
# the data and its copy
a = [[1, 2], [3, 4]]
b = copy.copy(a) # Shallow Copy
```

Gambar 3.7 Nested List dan Shallow Copy

```
# Modify the value at index [0][0] in b
b[0][0] = 99
print("Original data a: ", a)
print("Shallow copy b: ", b)
```

```
Original data a:  [[99, 2], [3, 4]]
Shallow copy b:  [[99, 2], [3, 4]]
```

Gambar 3.8 Hasil Shallow Copy

Shallow copy membuat objek container baru di dalam memori, tetapi untuk elemen-elemen yang berada di dalamnya, shallow copy hanya menyalin alamat referensinya. Mari kita buktikan dengan melakukan hal berikut:

```
print("Id of a in memory: ", id(a))
print("Id of b in memory: ", id(b))

print("\nAre IDs of a and b at index [0][0] identical?: ",
      id(a[0][0]) == id(b[0][0]))
```

```
Id of a in memory:  133013328758656
Id of b in memory:  133013328734016

Are IDs of a and b at index [0][0] identical?:  True
```

Gambar 3.9 Membuktikan Elemen di Dalamnya Berbagi Referensi yang Sama

Meskipun variabel a dan variabel b memiliki ID pada level teratas yang sepenuhnya berbeda, ketika memeriksa ID keduanya pada indeks nested [0][0], hasilnya tetap True. Output ini memberikan bukti yang jelas tentang bagaimana shallow copy bekerja di balik layar. Ketika menjalankan copy.copy(a), Python membuat container list bagian luar yang benar-benar baru di dalam memori, sehingga id(a) dan id(b) memiliki nilai yang berbeda. Namun, alih-alih menduplikasi data yang sebenarnya tersimpan di dalam container tersebut, Python menghemat waktu dengan hanya menyalin alamat referensi memori dari list nested di dalamnya.

Karena duplikasi yang hanya dilakukan pada lapisan luar ini, list asli a dan list hasil salinan b akhirnya menunjuk ke list bagian dalam yang sama di dalam memori. Referensi yang sama ini menimbulkan risiko tersembunyi yang besar dalam menjaga kestabilan data. Ketika kita memodifikasi elemen nested menggunakan b[0][0] = 99, perubahan tersebut akan langsung terlihat pada variabel a juga. Kedua

variabel tersebut tidak benar-benar terisolasi karena masih terhubung melalui lapisan bagian dalamnya.

Pada akhirnya, perilaku ini menunjukkan bahwa shallow copy hanya aman dan efektif ketika kita bekerja dengan struktur data yang datar dan satu dimensi. Ketika proyek mulai menggunakan data nested atau compound, seperti list di dalam list atau dictionary di dalam list, penggunaan shallow copy dapat secara tidak sengaja merusak isolasi yang diperlukan dalam Effect-Free Programming dan Pure Function.

Jadi, bagaimana cara menduplikasi semua lapisan dari nested list dengan aman? Jawabannya ada di bawah ini.

b. Deep Copy

Deep copy mengatasi masalah keterikatan referensi pada data nested (seperti nested list yang telah dibahas sebelumnya). Fungsi copy.deepcopy() menyalin objek secara menyeluruh dan rekursif hingga ke elemen terdalam, sehingga objek baru benar-benar terpisah sepenuhnya dari objek aslinya.

```
# Reset data and use a deepcopy() function
a = [[1, 2], [3, 4]]
c = copy.deepcopy(a)
```

Gambar 3.10 Nested List dan Deep Copy

```
c[0][0] = 100
print("Original data a:", a)
print("Copied data c:", c)
```

```
Original data a: [[1, 2], [3, 4]]
Copied data c: [[100, 2], [3, 4]]
```

Gambar 3.11 Hasil Deep Copy

Untuk memahami mengapa cara ini berhasil, perhatikan bagaimana Python menangani proses penyalinan secara rekursif. Alih-alih hanya menduplikasi container list bagian luar dan menyalin alamat referensi dari list di dalamnya, deepcopy menelusuri setiap lapisan struktur data. Python membuat container bagian luar yang benar-benar baru, lalu membuat salinan yang sepenuhnya terpisah dari setiap nested list di dalamnya. Karena setiap lapisan diduplikasi hingga ke elemen terdalam, objek baru tersebut 100% terpisah dari objek asli di memori komputer.

Pada akhirnya, modul copy sangat membantu dalam membangun Pure Function karena mencegah perubahan langsung

pada data asli, sehingga kode tetap mengikuti prinsip Effect-Free Programming. Ketika memilih antara shallow copy dan deep copy, pilihannya sepenuhnya bergantung pada struktur data yang digunakan. Untuk data sederhana satu dimensi (seperti flat list yang berisi integer), shallow copy sudah lebih dari cukup dan lebih cepat. Namun, jika data yang digunakan kompleks dan nested (seperti list yang berisi list atau dictionary lainnya), deep copy merupakan pilihan yang tepat untuk menjamin isolasi sepenuhnya dan melindungi dataset asli dari perubahan yang tidak disengaja.

📌 Exercise 2: Data duplication with shallow and deep copy

Selanjutnya, coba salin data berikut:

```
user = { "name" : "putri", "scores" : [90, 85]}
```

Figure 3.12 Exercise 2

Instruksi:

1. Buat duplikat dari variabel user menggunakan Shallow Copy (simpan dalam variabel bernama user_shallow) dan Deep Copy (simpan dalam variabel bernama user_deep).
2. Pada masing-masing objek hasil salinan (user_shallow dan user_deep), ubah nilai scores dari 90 menjadi 75.
3. Periksa alamat memori dari indeks yang telah dimodifikasi, lalu bandingkan dengan alamat memori pada variabel user asli.

📝 Analysis Question 2

Amati output data asli beserta alamat memorinya, kemudian jelaskan mengapa hasil Shallow Copy dan Deep Copy berbeda!

4. DECLARATIVE PROGRAMMING

Setelah memahami cara membangun Pure Function, langkah selanjutnya adalah mengenal konsep yang lebih luas dalam Functional Programming, yaitu Declarative Programming Paradigm. Pemahaman mengenai paradigma ini akan menjadi lebih jelas jika dibandingkan dengan pendekatan yang berlawanan, yaitu Imperative Programming Paradigm. Kedua paradigma ini memiliki perbedaan mendasar dalam cara menyelesaikan masalah. Mari kita lihat materi berikut.

a. Imperative Paradigm

Imperative Paradigm merupakan pendekatan pemrograman yang paling tradisional dan berakar pada cara kerja perangkat keras komputer. Dalam pendekatan ini, program dipandang sebagai serangkaian instruksi yang secara eksplisit mengatur perubahan state pada komputer. Alih-alih

menjelaskan sifat matematis dari suatu masalah, developer berperan sebagai pengawas yang mengarahkan komputer melalui urutan eksekusi yang ketat dan linier.

Untuk memahami konsep ini, mari kita lihat masalah dasarnya, yaitu menghitung jumlah total dari sekumpulan angka. Dalam pola pikir imperatif, kita tidak dapat langsung meminta hasil. Kita harus membuat sebuah "resep" yang menjelaskan secara rinci bagaimana hasil tersebut dibangun sejak awal. Perhatikan contoh kode Imperative Programming berikut:

```
# Create a list containing three numbers
numbers1 = [10, 15, 30]

# Initialize an accumulator variable to store the running total
total = 0

# Iterate through each element in the list
for i in numbers1:
        total += i # Add the current element to the accumulator

# Display the final result after all elements have been processed
print(total)
```

```
55
```

Gambar 4.1 Contoh Imperative

Dalam contoh ini, proses dimulai dengan membuat variabel accumulator bernama total dan menginisialisasinya dengan nilai 0. Program kemudian melakukan iterasi pada setiap elemen dalam list numbers1. Pada setiap iterasi, elemen saat ini ditambahkan ke accumulator menggunakan pernyataan total += i.

Untuk memahami tujuan program ini, kita perlu melihat dan mengikuti setiap langkah yang dilakukan oleh kode. Programmer secara eksplisit mengatur proses perhitungan, termasuk membuat variabel accumulator, mengelola loop, dan memperbarui total pada setiap iterasi. Fokusnya terletak pada urutan operasi yang diperlukan untuk mendapatkan hasil akhir (**how to do it**), yang merupakan **fundamental utama dari Imperative Programming.**

b. Declarative Paradigm

Declarative Programming memiliki sudut pandang yang berbeda dari Imperative Programming. Jika Imperative Programming mengharuskan programmer menjelaskan urutan operasi yang diperlukan untuk menyelesaikan suatu masalah (how to do it), **Declarative Programming berfokus pada menentukan hasil yang diinginkan (what to do)**. Pada contoh Imperative sebelumnya, kita harus melihat urutan pernyataan untuk memahami bahwa program tersebut digunakan untuk menghitung total.

Dalam Declarative Programming, developer sering berinteraksi dengan fungsi melalui spesifikasi atau kontraknya tanpa perlu memahami detail implementasi yang tersembunyi di dalamnya. Hal yang penting bagi pengguna adalah tujuan fungsi, input yang diperlukan, dan output yang diharapkan. Proses perhitungan di dalamnya dapat diabstraksikan.

Dengan demikian, kita sering kali dapat mengetahui apa yang dilakukan sebuah fungsi hanya dengan membaca spesifikasinya, tanpa perlu menganalisis implementasinya. Fokusnya bergeser dari memahami urutan operasi yang digunakan untuk menghasilkan suatu hasil menjadi memahami hasil yang ingin dihasilkan oleh fungsi tersebut. Perhatikan contoh berikut:

```
# Preparing declarative functions
'''
'total1' function
pre: take any sequence number 'x' as input
post: return the sum of all number in 'x'
'''
def total1(x):
    total = 0
    for i in numbers1:
        total += i
    return total
```

Gambar 4.2 Fungsi Declarative total1() Menggunakan Iterasi

```
# Preparing declarative functions
'''
'total2' function
pre: take any sequence number 'x' as input
post: return the sum of all number in 'x'
'''
def total2(x):
  n=len(x)-1
  def rekursi(x,n):
    if n == 0:
      return x[n];
    else:
      return x[n]+rekursi(x,n-1)
  return rekursi(x,n)
```

Gambar 4.3 Fungsi Declarative total2() Menggunakan Rekursi

```
# Preparing declarative functions
'''
'total3' function
pre: take any sequence number 'x' as input
post: return the sum of all number in 'x'
'''
def total3(x):
    return sum(x)
```

Gambar 4.4 Fungsi Declarative total3() Menggunakan sum()

Ketiga fungsi di atas menggunakan implementasi yang berbeda. Fungsi total1() menggunakan iterasi, total2() menggunakan rekursi, dan total3() menggunakan fungsi bawaan Python, yaitu sum(). Namun, ketiganya memiliki tujuan yang sama, yaitu "menghitung jumlah seluruh angka dalam sebuah sequence". Dengan input data yang sama, ketiga fungsi tersebut menghasilkan hasil yang sama seperti berikut:

```
# Leverage declarative functions to build programs
print('result with 1st function =', total1(numbers1))
print('result with 2nd function =', total2(numbers1))
print('result with 3rd function =', total3(numbers1))
```

```
result with 1st function = 55
result with 2nd function = 55
result with 3rd function = 55
```

Gambar 4.5 Perbandingan Output total1(), total2(), total3()

Ketika menggunakan fungsi-fungsi tersebut, pengguna tidak perlu memahami detail implementasinya. Dengan membaca spesifikasi fungsi:

```
'''
'total1' function
pre: take any sequence number 'x' as input
post: return the sum of all number in 'x'
'''
```

Gambar 4.6 Spesifikasi Fungsi

Kita dapat mengetahui bahwa fungsi ini menerima sequence angka sebagai input dan mengembalikan jumlah seluruh angka tersebut sebagai output. Dengan kata lain, fokus pengguna adalah pada apa yang dilakukan oleh fungsi, bukan pada bagaimana fungsi tersebut diimplementasikan. Implementasinya dapat berubah tanpa memengaruhi cara fungsi tersebut digunakan, selama kontrak fungsi tetap sama dan fungsi tersebut bersifat pure.

5. ITERATOR & GENERATOR

Setelah memahami paradigma Declarative Programming pada bagian sebelumnya, kita akan membahas dua instrumen penting dalam Python yang sering digunakan untuk menerapkan Declarative Programming dan Functional Programming tingkat lanjut, yaitu Iterator dan Generator. Kedua fitur ini berfokus pada efisiensi pemrosesan data, terutama ketika kita berhadapan dengan data dalam jumlah besar.

5.1. KONSEP ITERATOR

Iterasi merupakan proses melakukan suatu operasi secara berulang menggunakan logika yang sama, biasanya melalui loop seperti for dan while. Sementara itu, iterator merupakan objek yang memungkinkan elemen dari suatu kumpulan data, seperti list, tuple, atau set, diakses secara berurutan, satu per satu, hingga seluruh data telah diproses.

- **Iterasi** adalah proses menelusuri kumpulan data untuk melakukan operasi yang sama pada setiap elemen secara berurutan.
- **Iterable object** adalah container atau kumpulan data yang elemennya dapat ditelusuri, seperti list atau string.
- **Iterator** adalah mekanisme yang melakukan proses penelusuran tersebut. Iterator menyimpan posisi saat ini sehingga dapat mengambil elemen berikutnya dalam sequence.

Iterator bekerja menggunakan konsep lazy evaluation, yaitu mekanisme ketika nilai tidak dihitung atau diambil sekaligus saat iterator dibuat. Sebaliknya, proses perhitungan dan pengambilan nilai dilakukan hanya ketika nilai tersebut benar-benar dibutuhkan. Dengan kata lain, iterator menghasilkan atau menghitung nilai hanya ketika diminta oleh program. Pendekatan ini meningkatkan efisiensi memori dan mengurangi perhitungan yang tidak diperlukan karena nilai dihasilkan secara bertahap sesuai kebutuhan.

Analogi:
Bayangkan kamu sedang memeriksa daftar kehadiran. Kamu menelusuri daftar nama satu per satu dan memeriksa setiap orang secara berurutan. Proses menelusuri daftar secara sistematis tersebut merupakan iterasi. Proses ini juga dilakukan sesuai kebutuhan, satu per satu, bukan semuanya sekaligus. Inilah konsep lazy evaluation.

Untuk mengubah iterable object menjadi iterator, Python menyediakan fungsi bawaan bernama **iter()**. Sementara itu, untuk mengambil nilai berikutnya satu per satu, kita menggunakan fungsi bawaan **next()**. Perhatikan contoh berikut:

```python
# list_names is an Iterable object, but it is NOT an Iterator
list_names = ["Amel", "Rani", "Charlie"]
```

Gambar 5.1 Menentukan Nilai list_name

```python
# Convert the Iterable into an Iterator object using iter() function
names_iterator = iter(list_names)

# some test of iterator
print("Object Type:", type(names_iterator))
# Output: <class 'list_iterator'>

# Retrieve values one by one using next()
print(next(names_iterator)) # Output: Amel
print(next(names_iterator)) # Output: Rani
print(next(names_iterator)) # Output: Charlie
```

```
Object Type: <class 'list_iterator'>
Amel
Rani
Charlie
```

Gambar 5.2 Mengubah List Menjadi Iterator dengan iter() & Mengambil Nilai dengan next()

Untuk menghindari kesalahpahaman, kita perlu melihat lebih dekat apa yang terjadi di memori komputer selama proses ini. List biasa seperti list_names merupakan iterable object, yang berarti list tersebut menyimpan kumpulan nilai yang dapat ditelusuri. Namun, iterable tidak selalu merupakan iterator. Sebagai contoh, jika kita langsung memanggil next(list_names), Python akan menghasilkan TypeError karena sebuah list tidak menyimpan state atau posisi penelusurannya sendiri.

Ketika kita menjalankan names_iterator = iter(list_names), Python mengubah iterable object menjadi iterator dengan membuat objek pembungkus baru bernama list_iterator. Iterator ini tidak menduplikasi data asli. Sebaliknya, iterator menyimpan pointer referensi internal yang melacak posisi saat ini dalam sequence. Setelah objek diubah menjadi iterator, elemen-elemennya dapat diakses secara berurutan menggunakan fungsi next(). Dengan demikian, nilai hanya diambil ketika diminta oleh program. Perilaku ini mengikuti konsep lazy evaluation, yaitu data diproses sesuai kebutuhan, bukan sekaligus.

Penting untuk diperhatikan bahwa list asli mungkin sudah sepenuhnya tersimpan di dalam memori, tetapi konsep lazy di sini mengacu pada bagaimana data diakses, bukan bagaimana data tersebut disimpan.

Setiap kali next(names_iterator) dipanggil, terdapat dua operasi yang terjadi secara internal:

1. Iterator memajukan pointer internalnya ke posisi berikutnya yang tersedia.
2. Nilai pada posisi tersebut diambil dan dikembalikan kepada pemanggil.

Karena iterator mempertahankan state yang terus berubah di memori, iterator bekerja seperti mekanisme penelusuran satu arah. Iterator hanya dapat bergerak maju melalui sequence dan tidak dapat secara otomatis kembali ke posisi sebelumnya. Setelah elemen terakhir ("Charlie") diambil, iterator mencapai akhir sequence. Pemanggilan next(names_iterator) berikutnya akan menghasilkan exception StopIteration yang memberi tahu Python bahwa tidak ada nilai yang tersisa untuk diproses dan proses iterasi telah selesai. Coba jalankan kode berikut:

```
# If called once more,
# it will trigger a StopIteration error because there is no data left

print(next(names_iterator))
```

Gambar 5.3 Memicu StopIteration

📌 Exercise 3: Using iter() and next() to Process Tuple Data

Buat sebuah tuple bernama sensor_data = (23.5, 24.0, 22.8, 21.1). Ubah tuple tersebut menjadi objek iterator menggunakan fungsi iter(). Ambil dan tampilkan dua nilai pertama dari iterator menggunakan fungsi next(), kemudian tampilkan data yang tersisa dan belum diambil menggunakan for loop biasa!

📝 Analysis Question 3

1. Amati output dari for loop. Mengapa for loop langsung menampilkan data yang tersisa (data ketiga) alih-alih melakukan iterasi dari elemen pertama?
2. Jelaskan bagaimana mekanisme for loop bekerja di balik layar dalam Python saat menangani objek iterator tanpa menggunakan fungsi next() secara eksplisit?

5.2. LIST COMPREHENSION

Ketika bekerja dengan dataset berukuran besar, mendefinisikan setiap elemen dalam sebuah list secara manual tidak praktis dan sulit untuk dipelihara. Python menyediakan fitur khusus untuk menyederhanakan proses ini yang disebut List Comprehension. List Comprehension merupakan cara yang ringkas dan deklaratif untuk membuat list baru dari iterable object (seperti range, list, atau string) dalam satu baris kode.

Sintaks umumnya ditulis sebagai berikut:

new_list = [expression for item in iterable if condition]

Penjelasan:

- expression: Nilai atau hasil yang akan dimasukkan ke dalam list baru (berperan sebagai elemen yang akan ditambahkan).
- item: Variabel yang merepresentasikan setiap elemen selama proses iterasi berlangsung.
- iterable: Sumber data yang dapat di-loop, seperti range, list, atau string.
- condition: Syarat tambahan (opsional) yang digunakan untuk memfilter dan memilih elemen tertentu.

```
# List comprehension example
new_list = [x * 10 for x in range(1, 101) if x % 2 == 0]

print(type(new_list)) #Output: <class 'list'>
print(new_list)
# Output: [20, 40, 60, 80, 100, 120, 140, 160, 180, 200, ..., 1000]
```

```
<class 'list'>
[20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260, 280, 300, 320,
```

Gambar 5.4 Contoh List Comprehension

Komponen-komponen dalam contoh ini adalah:

1. Expression: x * 10; setiap nilai yang dipilih dikalikan dengan 10 sebelum dimasukkan ke dalam list baru.
2. item: x, variabel x merepresentasikan setiap elemen yang diperoleh selama proses iterasi.
3. Iterable: range(1, 101). Fungsi range(1, 101) menghasilkan angka dari 1 hingga 100 yang digunakan sebagai sumber data untuk iterasi.
4. condition: x % 2 == 0; hanya angka genap yang memenuhi kondisi ini. Angka ganjil diabaikan dan tidak dimasukkan ke dalam daftar baru.
5. Hasilnya, List Comprehension melakukan iterasi pada angka dari 1 hingga 100, memilih hanya angka genap, mengalikan setiap angka yang dipilih dengan 10, dan menyimpan hasilnya ke dalam list baru.

Selanjutnya, kita akan mempelajari bagaimana memanfaatkan List Comprehension saat bekerja dengan data multidimensi seperti matriks m × n, sehingga kita dapat memanipulasi struktur data yang kompleks secara efisien dan tetap mudah dipahami.

Nested List Comprehension:

newlist = [[expression for item in inner_iterable] for item in outer_iterable]

Penjelasan:

- List bagian dalam ([expression for item in inner_iterable]) menghasilkan sublist.
- List bagian luar (for item in outer_iterable) mengulangi proses pembuatan list bagian dalam dan menggabungkannya menjadi list utama.

Perhatikan kode berikut:

```
numbers = [
    [1, 2, 3],
    [4, 5, 6]
]

# Add 1 to Every Element
new_list = [[num + 1 for num in row] for row in numbers]

print(new_list)
```

```
[[2, 3, 4], [5, 6, 7]]
```

Gambar 5.5 Nested List Comprehension

Cara kerjanya secara bertahap:

1. Outer loop (for row in numbers)
    - Melakukan iterasi pada setiap sublist dalam numbers.
    - Pada iterasi pertama, row = [1, 2, 3].
    - Pada iterasi kedua, row = [4, 5, 6].
2. Inner loop (for num in row)
    - Melakukan iterasi pada setiap elemen dalam sublist saat ini.
    - Sebagai contoh, ketika row = [1, 2, 3], nilai num adalah 1, 2, dan 3.
3. Expression (num + 1)
    - Menambahkan 1 pada setiap nilai num.
4. Inner List Comprehension ([num + 1 for num in row])
    - Membuat sublist baru dengan menerapkan expression pada setiap elemen dalam row.
    - Sebagai contoh: [num + 1 for num in [1, 2, 3]] menghasilkan [2, 3, 4].
5. Outer List Comprehension ([... for row in numbers])
    - Mengumpulkan setiap sublist yang telah dibuat ke dalam nested list baru dengan tetap mempertahankan struktur dua dimensi.

📌 Exercise 4: Convert to List Comprehension

Ubah kode berikut menjadi List Comprehension:

```
result = []
for letter in "universitas":
    if letter not in "aiueo":
        result.append(letter.upper())

print(result)
```

```
['N', 'V', 'R', 'S', 'T', 'S']
```

Gambar 5.6 Exercise 4

!! Small Experiment !!

Sekarang, mari kita modifikasi contoh sebelumnya dengan mengganti tanda kurung siku **[ ]** dengan tanda kurung biasa **( )**. Jalankan kode tersebut, amati output-nya, lalu analisis apakah tipe objek yang dihasilkan berubah.

```
# Modifiy this and observe
new_list = [x * 10 for x in range(1, 101) if x % 2 == 0]

print(type(new_list))
print(new_list)
```

```
<class 'list'>
[20, 40, 60, 80, 100, 120, 140, 160, 180, 200, 220, 240, 260, 280, 300, 32
```

Gambar 5.7 Mengganti Tanda Kurung Siku dengan Tanda Kurung Biasa

Hasilnya mungkin berbeda dari yang kamu perkirakan sebelumnya. Untuk memahami mengapa hal ini terjadi, kita perlu mempelajari Generator Expressions pada bagian berikutnya!

5.3. GENERATOR EXPRESSION

Bayangkan aplikasi kamu perlu memproses log server berukuran hingga ratusan gigabyte. Jika seluruh dataset dimuat ke dalam memori sekaligus (misalnya dengan membuat list besar menggunakan List Comprehension), program kemungkinan akan kehabisan memori dan mengalami crash. Generator Expression mengatasi masalah ini dengan menghasilkan elemen sesuai kebutuhan, bukan dengan menyimpan seluruh kumpulan data di memori. Untuk menghasilkan perilaku yang hemat memori ini, Python menyediakan sintaks yang sangat mirip dengan List Comprehension.

Sintaks Generator Expression sangat mirip dengan List Comprehension, dengan satu perbedaan utama, yaitu jenis tanda kurung yang digunakan:

- List Comprehension: Menggunakan tanda kurung siku **[...]**
- Generator Expression: Menggunakan tanda kurung **(...)**

Sekilas, kamu mungkin mengira bahwa mengganti tanda kurung siku dengan tanda kurung biasa akan menghasilkan tuple. Namun, dalam Python, tanda kurung yang digunakan dalam comprehension akan menghasilkan Generator Expression yang menghasilkan generator object, bukan tuple. Hal ini karena generator dirancang untuk memproses data dengan lebih efisien dalam penggunaan memori. Berbeda dengan list yang langsung menghasilkan dan menyimpan semua elemen setelah dibuat, generator hanya menghasilkan nilai saat diminta selama proses iterasi, seperti pada konsep iterator. Perilaku ini dikenal sebagai lazy evaluation.

Dengan demikian, Generator Expression dapat memproses data dalam jumlah besar dengan menggunakan memori yang jauh lebih sedikit dibandingkan List Comprehension, sehingga sangat sesuai untuk aplikasi yang membutuhkan banyak memori.

```
# List comprehension implementation
numbers_list = [x * 10 for x in range(1, 4)]
print("List Result:", numbers_list)
# Direct data output: [10, 20, 30]

# Generator expression implementation
numbers_gen = (x * 10 for x in range(1, 4))
print("Gen Result :", numbers_gen)
# Output is a memory address reference: <generator object ...>

# Accessing the generator expression
print("\nGenerator Expression Output:")
for number in numbers_gen:
    print(number, end=" ") # Output: 10 20 30
```

```
List Result: [10, 20, 30]
Gen Result : <generator object <genexpr> at 0x78cca6ec39f0>

Generator Expression Output:
10 20 30
```

Gambar 5.8 List Comprehension vs Generator Expression

Ketika kedua variabel dicetak secara langsung, kamu akan melihat perbedaan yang jelas dalam cara kerjanya. List Comprehension menjalankan seluruh loop secara langsung, mengisi list secara lengkap di dalam memori, dan menghasilkan output [10, 20, 30]. Sementara itu, ketika numbers_gen dicetak, hasilnya berupa referensi memori seperti <generator object <genexpr> at 0x78f994480ad>. Hal ini terjadi karena Generator Expression menghasilkan data secara lazy. Generator tidak langsung

menghitung atau menyimpan nilai saat dideklarasikan. Sebaliknya, generator menunggu hingga for loop meminta setiap elemen secara eksplisit. Perilaku sesuai kebutuhan ini merupakan contoh nyata dari konsep fundamental dalam Functional Programming yang dikenal sebagai Lazy Evaluation.

📌 **Exercise 5: Create Generator Expression**

Buat sebuah Generator Expression yang menghasilkan angka dari 1 hingga 5, lalu tampilkan nilainya menggunakan next() di dalam while loop. Tambahkan exception handling untuk menangani kondisi ketika data sudah habis (StopIteration) dan tampilkan pesan yang menunjukkan bahwa iterator telah selesai.

5.4. GENERATOR

Generator Expression menyediakan cara yang ringkas untuk membuat generator object menggunakan sintaks satu baris. Generator Expression sangat berguna untuk transformasi data sederhana dan operasi filtering. Namun, tidak semua proses pembentukan data dapat ditulis dengan mudah dalam satu expression. Dalam berbagai kasus nyata, kita mungkin membutuhkan logika yang lebih kompleks yang melibatkan beberapa kondisi, loop, atau perhitungan perantara. Untuk mengatasi situasi tersebut, alih-alih mendefinisikan generator menggunakan satu expression yang diapit tanda kurung, kita dapat membuat Generator Function menggunakan keyword def bersama dengan pernyataan **yield**.

Generator merupakan jenis fungsi khusus yang menghasilkan nilai secara bertahap dan iteratif. Sementara fungsi biasa menggunakan keyword return untuk mengembalikan nilai dan mengakhiri konteksnya sepenuhnya, Generator Function menggunakan keyword yield. Ketika komputer menemukan pernyataan yield, fungsi akan berhenti sementara untuk menghasilkan nilai dan mempertahankan state eksekusinya hingga fungsi dipanggil kembali.

Ketika nilai berikutnya diminta, fungsi akan melanjutkan eksekusi dari posisi tempat fungsi tersebut berhenti sebelumnya. Proses ini terus berlangsung hingga seluruh nilai telah dihasilkan. Dengan demikian, Generator Function mendukung lazy evaluation dan dapat memproses dataset berukuran besar secara efisien tanpa harus menyimpan seluruh nilai dalam memori sekaligus. Perhatikan contoh kode berikut:

```
"""
    'squared_generator' function
pre: take any number as 'limit'
post: Generates squared numbers from 1 to
the specified limit as generator object
"""
def squared_generator(limit):
    n = 1
    while n <= limit:
        yield n ** 2
        n += 1
```

Gambar 5.9 Generator Function dengan yield

```
# generator object
number_output = squared_generator(3)

print("Object Type:", type(number_output))
# Output: <class 'generator'>
print("Results:")
for x in number_output:
    print(x)
```

```
Object Type: <class 'generator'>
Results:
1
4
9
```

Gambar 5.10 Output Generator yang Dipanggil Secara Bertahap

Fungsi tersebut tidak mengembalikan seluruh data sekaligus, tetapi menghasilkan nilai satu per satu setiap kali dipanggil. Kunci dari perilaku ini terletak pada perbedaan cara Python menangani return dan yield. Ketika fungsi biasa menjalankan pernyataan return, konteks eksekusinya sepenuhnya dihentikan dan seluruh variabel lokal dihapus dari memori. Sebaliknya, ketika Generator Function menemukan pernyataan yield, Python menangguhkan sementara eksekusi fungsi, tetapi tetap mempertahankan seluruh state lokalnya, termasuk nilai variabel n saat ini dan posisi instruksi berikutnya.

Ketika loop meminta nilai berikutnya, Python tidak menjalankan fungsi dari awal. Sebaliknya, Python melanjutkan fungsi tepat dari baris setelah yield terakhir yang dijalankan. Loop kemudian menambah nilai n dan mengevaluasi kembali kondisi tersebut. Siklus penangguhan dan pelanjutan ini terus berlangsung hingga kondisi while tidak lagi terpenuhi, kemudian generator akan berhenti secara otomatis. Dengan

mengevaluasi expression hanya ketika diminta secara langsung, generator berperan sebagai aliran data yang sangat efisien karena menghitung nilai sesuai kebutuhan.

📌 Exercise 6: Create Generator

Skenario: Sebuah server mencatat aktivitas sign-in menggunakan format string tertentu. Dalam lingkungan produksi, file log dapat berkembang hingga berukuran gigabyte, sehingga data tidak boleh dimuat sepenuhnya ke dalam RAM menggunakan List. Kita perlu membuat fungsi "lazy" untuk memfilter hanya log darurat.

Diberikan data raw log stream berikut (disimulasikan):

```
server_logs = [
    "INFO: User login successful",
    "ERROR: Database connection failed",
    "INFO: Page rendered in 25ms",
    "ERROR: Unauthorized access attempt",
    "INFO: User logout successful"
]
```

Gambar 5.11 Data yang Digunakan

Instruksi:

1. Buat Generator Function bernama filter_emergency_log(log_list) yang menerima data log.
2. Fungsi harus mengevaluasi setiap baris log. Jika sebuah baris log diawali dengan keyword "ERROR", hasilkan (yield) baris log tersebut secara bertahap.
3. Pada program utama, panggil Generator Function tersebut dan simpan hasilnya ke dalam variabel bernama error_bucket.
4. Ambil nilai pertama dan kedua secara manual menggunakan fungsi next().

6. LAZY EVALUATION

Pada pembahasan sebelumnya, kita telah melihat bahwa generator dapat menghasilkan data secara berurutan. Perilaku ini berkaitan erat dengan konsep penting dalam Functional Programming, yaitu Lazy Evaluation. Lazy Evaluation merupakan teknik ketika data tidak dibuat sekaligus, tetapi hanya dihasilkan ketika benar-benar dibutuhkan. Dengan kata lain, program tidak memproses seluruh data pada awalnya, tetapi menunggu hingga bagian data tertentu diminta. Untuk memahami manfaat teknik ini, kita dapat membandingkannya dengan Eager Evaluation, yaitu ketika komputer secara langsung menghitung dan mengalokasikan memori untuk setiap elemen segera setelah baris kode dibaca. Untuk memahami hal ini dengan lebih baik, perhatikan kode berikut:

```python
from sys import getsizeof

# Eager Evaluation using list comprehension
data_eager = [x * 2 for x in range(1, 100000)]

# Lazy Evaluation useing generator expression
data_lazy = (x * 2 for x in range(1, 100000))
```

Gambar 6.1 Definisi Data Eager vs Lazy

```python
# Performing type
print("Eager Object Type :", type(data_eager))
print("Lazy Object Type  :", type(data_lazy))
```

```
Eager Object Type : <class 'list'>
Lazy Object Type  : <class 'generator'>
```

Gambar 6.2 Memeriksa Tipe Objek (list vs generator)

```python
# Show size of each object
print(f"Size of Eager Object (List)      : {getsizeof(data_eager)} Bytes")
print(f"Size of Lazy Object (Generator)  : {getsizeof(data_lazy)} Bytes")
```

```
Size of Eager Object (List)      : 800984 Bytes
Size of Lazy Object (Generator)  : 200 Bytes
```

Gambar 6.3 Membandingkan Ukuran Memori dengan getsizeof()

```python
# Show data of each object
print("Eager Object :", data_eager)
print("Lazy Object  :", data_lazy)
```

```
Eager Object : [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32,
Lazy Object  : <generator object <genexpr> at 0x78cca6ec3ed0>
```

Gambar 6.4 Memeriksa Isi Data

Penggunaan memori pada kedua objek tersebut berbeda secara signifikan. List menggunakan memori yang jauh lebih besar karena Python langsung menghasilkan dan menyimpan setiap nilai saat objek dibuat. Perilaku ini dikenal sebagai Eager Evaluation, yaitu seluruh perhitungan dilakukan di awal tanpa mempertimbangkan apakah setiap nilai nantinya benar-benar akan digunakan. Sebaliknya, generator hanya menggunakan sedikit memori karena tidak menghasilkan semua nilai secara langsung.

Generator hanya menyimpan instruksi yang diperlukan untuk menghasilkan nilai berikutnya. Nilai sebenarnya dihasilkan satu per satu hanya ketika diminta selama proses iterasi. Pendekatan ini dikenal sebagai Lazy Evaluation. Alih-alih melakukan seluruh perhitungan sekaligus, program menunda perhitungan hingga hasil tersebut benar-benar dibutuhkan. Pendekatan ini terbukti jauh lebih efisien dalam eksekusi dan sangat ringan dalam penggunaan sumber daya sistem.

📝 Analysis Question 4

Dari Exercise 6, jika volume data server log meningkat menjadi 10.000.000 baris, mengapa penggunaan Generator Function dengan yield jauh lebih aman untuk memori komputer dibandingkan dengan fungsi biasa yang mengumpulkan seluruh hasil ke dalam list dan mengembalikannya sekaligus menggunakan return?

CODE ANALYSIS

Tugas Code Analysis terdiri dari Exercise (1 sampai 6) dan Analysis Questions. Selesaikan exercises pada Notebook, dan tulis jawabanmu di *Template laporan yang disediakan.*

Verifikasi & Deadline

5. Praktikan **wajib** sudah mengumpulkan laporan di iLab/MiLab **sebelum praktikum** pekan materi dimulai untuk dinilai/direvisi oleh asisten.
6. Praktikan **wajib menunjukkan** pengerjaan Exercise pada Notebook yang sedang berjalan dan memodifikasi Exercise sesuai dengan instruksi yang diberikan oleh asisten.
7. Jika tidak menunjukkan exercise dan tidak mengumpulkan laporan sesuai deadline, praktikan **tetap wajib mengumpulkan** laporan ke iLab/miLab dalam format **PDF** paling lambat **1 hari setelah praktikum (H+1)**. Namun, **Nilai Laporan Praktikum -40 poin dan mendapat nilai 0 untuk Exercise Implementation**.
8. Jika terdapat revisi yang diberikan oleh asisten, baik pada Exercise maupun Explanation Report, **praktikan wajib menyelesaikannya selama sesi praktikum berlangsung. Tidak ada penerimaan hasil revisi pada h+1 atau di iLab**.

ASSIGNMENT

ASSIGNMENT 1

Dalam tugas ini, kamu akan melakukan “Live Coding” seperti pada Modul 1. Untuk melaksanakannya, kamu akan diberikan link Colab oleh asisten selama demo dengan dataset yang berbeda. Persiapkan diri kamu dengan melakukan latihan pada Modul 2 di Colab.

ASSIGNMENT 2

Tujuan Pembelajaran

Setelah menyelesaikan tugas ini, mahasiswa diharapkan mampu:

1. Mengidentifikasi fungsi yang menghasilkan side effect.
2. Melakukan refactoring pada fungsi yang sudah ada dan membuat fungsi baru menjadi Pure Function.
3. Menerapkan prinsip Immutability dalam mengelola data (kamu dapat menggunakan modul copy untuk menghindari perubahan langsung pada data aslinya).
4. Menerapkan Declarative Programming.
5. Menggunakan List Comprehension untuk memanipulasi data.
6. Menjelaskan proses refactoring dan memberikan alasan atas perubahan yang dilakukan.

Tugas Utama

Gunakan program yang telah dikembangkan pada Assignment 2 - Module 1 sebagai dasar untuk tugas ini. Tugas kamu adalah melakukan refactoring pada kode yang sudah ada agar lebih mengikuti prinsip-prinsip Functional Programming, dengan tetap mempertahankan seluruh fitur dan fungsi dari Modul 1 tanpa menambahkan fitur baru atau mengubah tema proyek. Sebagai bagian dari proses refactoring, seluruh data akun, profil, dan data utama yang sebelumnya dibuat melalui operasi I/O (input()/print()), seperti pada fitur Register, harus diubah menjadi static dataset (misalnya menggunakan dictionary atau list of dictionaries). Data akun/profil dan data utama masing-masing harus memiliki minimal 10 data. Seluruh data yang dibutuhkan harus diambil dari static dataset tersebut.

Persyaratan Refaktorisasi Fungsional

1. Pure Function Refactoring
   Identifikasi fungsi yang masih menghasilkan side effect, seperti memodifikasi variabel global atau mengubah data secara langsung di luar fungsi. Lakukan refactoring pada fungsi-fungsi tersebut agar lebih mendekati Pure Function, dengan ketentuan:
   - Hasil fungsi hanya bergantung pada input yang diberikan.
   - Fungsi tidak mengubah data eksternal secara langsung.
   - Meminimalkan side effect yang tidak diperlukan.

Catatan: Fungsi yang menggunakan input() dan print() tidak dianggap sebagai Pure Function. Sebagai contoh, ketika membuat data baru menggunakan fungsi yang memiliki operasi I/O, kamu perlu melakukan refactoring atau membuat fungsi baru yang sepenuhnya pure tanpa operasi I/O. Dengan demikian, **business logic utama harus dipisahkan dari operasi input dan output.**

2. Immutability dan Modul copy
   Terapkan prinsip Immutability untuk menggantikan operasi yang secara langsung mengubah struktur data yang sudah ada. Untuk itu, pastikan data asli tidak berubah.
3. Declarative Programming
   Lakukan refactoring pada program agar lebih sesuai dengan gaya Declarative Programming.
   Contoh refactoring secara declarative dapat meliputi:
   - Mendefinisikan spesifikasi fungsi yang jelas (**precondition** dan **postcondition**).
   - Menyederhanakan proses pengolahan data menjadi expression yang ringkas dan dapat digunakan kembali.

   Minimal 2 bagian kode harus direfactor menggunakan pendekatan yang lebih declarative. Salah satu refactoring tersebut wajib menggunakan List Comprehension. Mahasiswa harus mampu menjelaskan mengapa solusi hasil refactoring dianggap lebih declarative dibandingkan implementasi sebelumnya.

ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
Code Analysis	**20**
Exercise Implementation	10
Practicum Report	10
Assignment 1	**40**
Code Completeness	20
Understanding	20
Assignment 2	**40**
Pure Function & Immutability	10
Declarative Programming	10
Understanding	20

Detailed Assessment Rubric

Code Analysis	Points (Total 20%)
Exercise Implementation	**10**
Seluruh exercise selesai dikerjakan dengan benar (poin akumulasi dari seluruh exercise).	100
Modifikasi kode tidak berhasil diselesaikan dengan benar	-50% (setiap exercise)
Exercise dikerjakan, namun kode/output salah	-30% (setiap exercise)
Jawaban tidak mengandung identitas praktikan (Nama/NIM)	-20% (setiap exercise)
Selesai melebihi jam praktikum	-100% (setiap exercise)
Practicum Report	**10**
Menggunakan template resmi, seluruh bagian explanation diisi dengan jawaban yang benar dan jelas	100
Jawaban salah atau jawaban kurang tepat	-50% (setiap exercise)
Penjelasan jawaban ditemukan identik dengan praktikan lain	-100% (setiap exercise)
Report dikirim melebihi deadline (saat praktikum)	-40 point (dari nilai akhir)
Assignment 1	**Points (Total 40%)**
Code Completeness	**20%**
Ketiga komponen berhasil diimplementasikan dengan benar, yaitu immutability data, generator expression, dan list comprehension.	100
Hanya dua komponen yang berhasil diimplementasikan dengan benar.	80
Hanya satu komponen yang berhasil diimplementasikan dengan benar.	60

Terdapat upaya untuk mengimplementasikan persyaratan, tetapi implementasinya tidak sesuai.	40
Tidak ada upaya untuk mengimplementasikan ketiga komponen tersebut.	0
Understanding	**20%**
Menjelaskan tujuan deepcopy() (1), cara kerja lazy evaluation pada generator (2), dan alasan validasi tipe data diperlukan dalam list comprehension (3).	100
Menjelaskan sebagian besar konsep yang diperlukan, tetapi kurang detail atau kurang mendalam pada salah satu bagian.	80
Memahami konsep secara umum, tetapi sebagian besar penjelasan masih kurang rinci.	60
Kesulitan menjelaskan sebagian besar kode dan konsep yang digunakan.	40
Tidak dapat menjelaskan konsep yang digunakan.	0
Assignment 2	**Points (Total 40%)**
Pure Function & Immutability	**10%**
Berhasil mengidentifikasi seluruh side effect yang tersembunyi dan melakukan refactoring pada fungsi impure menjadi transformasi yang lebih pure dan deterministik. Modul copy digunakan dengan tepat sebelum memodifikasi data, serta dilakukan verifikasi untuk memastikan bahwa data asli tetap tidak berubah setelah fungsi dijalankan.	100
Berhasil mengidentifikasi side effect utama dan melakukan refactoring pada fungsi untuk menghindari perubahan state secara langsung. Modul copy digunakan dengan benar dalam hampir semua kasus, meskipun masih terdapat sedikit kekurangan pada aspek purity, penyalinan data, atau verifikasi bahwa data asli tetap tidak berubah.	80
Kesulitan mengidentifikasi side effect yang tidak terlihat secara langsung, sehingga beberapa fungsi masih mengubah argumen input atau bergantung pada state eksternal. Penggunaan copying data juga tidak konsisten atau menggunakan copy.copy() alih-alih copy.deepcopy() untuk data bertingkat (nested data).	60

Tidak mampu membedakan pure function dan impure function dengan baik. Terdapat upaya menggunakan copying data atau melakukan refactoring, tetapi masih terdapat side effect yang signifikan serta perubahan yang tidak diinginkan pada data asli.	40
Tidak melakukan upaya untuk mengubah fungsi menjadi pure function atau menjaga data asli agar tetap tidak berubah. Modul copy tidak digunakan dan data asli mengalami perubahan selama eksekusi fungsi.	0
Declarative Programming	**10%**
Lebih dari 3 bagian kode menggunakan pendekatan declarative dengan List Comprehension.	100
≥ 2 bagian kode menggunakan pendekatan declarative, tetapi tidak menggunakan List Comprehension.	80
≤3 bagian kode menggunakan pendekatan declarative dan menggunakan List Comprehension.	60
Kurang dari 3 bagian kode menggunakan pendekatan declarative atau tidak menggunakan List Comprehension.	40
Tidak melakukan refactoring dengan pendekatan deklaratif.	0
Understanding	**20%**
Menjelaskan seluruh proses refactoring dengan jelas serta memberikan alasan yang tepat untuk setiap perubahan yang dilakukan.	100
Dapat menjelaskan sebagian besar proses refactoring, tetapi beberapa alasan yang diberikan masih belum lengkap.	80
Memahami konsep secara umum, tetapi alasan yang diberikan masih kurang mendalam.	60
Kesulitan menjelaskan proses refactoring serta alasan perubahan yang dilakukan.	40
Tidak dapat menjelaskan konsep dan proses refactoring.	0